

### Check differences

The state snapshot from the observe phase is then compared to the expected static cluster parameters specified in the Kubernetes configuration. Any discrepancies between the current state and the expected cluster state are identified and slated for action.

### Take action

The main then issues commands to bring the cluster back up to the expected state. This can mean removing or creating pods, horizontal or vertical scaling across nodes, and more. The above three phases are general descriptions of the Kubernetes loop. However, a Kubernetes cluster may actually have many different loops. These loops are executed by controllers. The ReplicationController or RC follows the loop phases. During the take action phase, RC is solely responsible for culling or creating new pods managed by the replica set.

## Creation of Kubernetes clusters

A Kubernetes cluster is a set of nodes that runs containerised applications. Containerisation of applications packages an app with its dependencies and some necessary services. Kubernetes clusters allow containers to run across multiple machines and environments — virtual, physical, cloud based, and on-premises. They are not restricted to a specific OS, and are able to share the OS run anywhere.

A Kubernetes cluster consists of two types of resources:

- The control plane coordinates the cluster (it is responsible for managing the cluster).
- Nodes are the workers that run applications (a node is

a VM or a physical computer that serves as a worker machine in a Kubernetes cluster).

Figure 1 explains the Kubernetes basic model, which includes creating Kubernetes, deploying an app, exploring the app, exposing the app publicly, as well as scaling up and updating the app.

## App installation

After running the Kubernetes cluster one can deploy containerised applications on top of it. To create and manage a deployment by using the Kubernetes command line interface, *kubectl*, the Kubernetes API, is used to interact with the cluster.

## Inspecting the application and deployment

In the deployment phase of Kubernetes, it is necessary to know about pods and nodes. A pod is a group of one or more application containers (such as Docker) and includes shared storage (volumes), the IP address, and information about how to run them.

The most common operations can be done with the following *kubectl* commands.

- *kubectl get*: List resources
- *kubectl describe*: Show detailed information about a resource
- *kubectl logs*: Print the logs from a container in a pod
- *kubectl exec*: Execute a command on a container in a pod

## Scaling and updating the application

Scaling is accomplished by changing the number of replicas in a deployment. It increases the number of pods to the new desired state. Kubernetes also supports auto-scaling of pods.

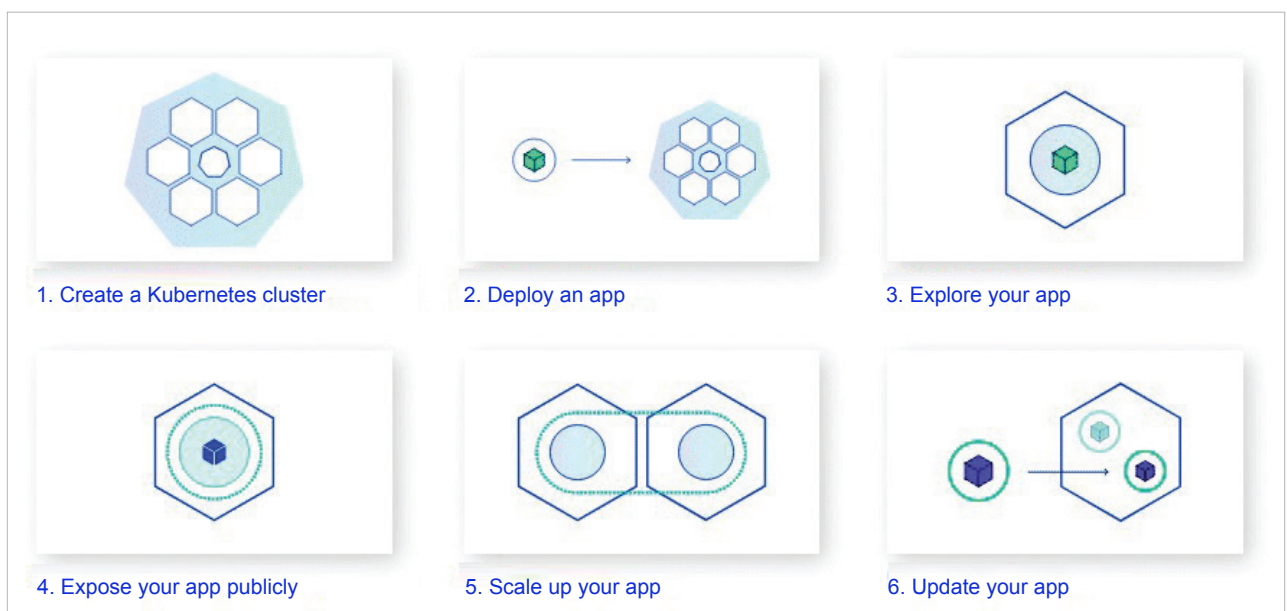


Figure 1: Basic modules of Kubernetes